

lab3_LI

Victor Li 197796

```
library(dplyr)
```

```
'dplyr'
```

```
'package:stats':
```

```
filter, lag
```

```
'package:base':
```

```
intersect, setdiff, setequal, union
```

```
library(tidyr)
```

```
library(readr)
```

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
```

```
v forcats 1.0.1      v purrr 1.2.2
```

```
v ggplot2 4.0.3      v stringr 1.6.0
```

```
v lubridate 1.9.5    v tibble 3.3.1
```

```
-- Conflicts ----- tidyverse_conflicts() --
```

```
x dplyr::filter() masks stats::filter()
```

```
x dplyr::lag() masks stats::lag()
```

```
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

Importe, utilizando o pacote readr, cada um dos três arquivos disponíveis. Os objetos resultantes devem ser chamados flights, airlines e airports. Para o arquivo de aeroportos, importe apenas as colunas IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE;

```
airports <- read_csv("airports.csv",
                     col_select = c(IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE))
```

Rows: 322 Columns: 5

-- Column specification -----

Delimiter: ","

chr (3): IATA_CODE, CITY, STATE

dbl (2): LATITUDE, LONGITUDE

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
airports
```

A tibble: 322 x 5

	IATA_CODE	CITY	STATE	LATITUDE	LONGITUDE
	<chr>	<chr>	<chr>	<dbl>	<dbl>
1	ABE	Allentown	PA	40.7	-75.4
2	ABI	Abilene	TX	32.4	-99.7
3	ABQ	Albuquerque	NM	35.0	-107.
4	ABR	Aberdeen	SD	45.4	-98.4
5	ABY	Albany	GA	31.5	-84.2
6	ACK	Nantucket	MA	41.3	-70.1
7	ACT	Waco	TX	31.6	-97.2
8	ACV	Arcata/Eureka	CA	41.0	-124.
9	ACY	Atlantic City	NJ	39.5	-74.6
10	ADK	Adak	AK	51.9	-177.

i 312 more rows

Para o arquivo de vôos: Importe apenas as colunas DESTINATION_AIRPORT e ARRIVAL_DELAY; a. Leia apenas 1 milhão de linhas por vez; b. Remova vôos em que o aeroporto de destino comece com a letra 1; c. Remova registros em que existam pelo menos uma coluna faltante; d. Determine, para cada parte do arquivo, as estatísticas suficientes para a determinação do atraso médio por aeroporto de destino; e. Ao finalizar a leitura do arquivo, combine as estatísticas suficientes de modo a produzir a média de atraso por aeroporto.

```
funcao <- function(input, put){
  input |>
    filter(!startsWith(DESTINATION_AIRPORT,"1"))|>
    filter(!is.na(DESTINATION_AIRPORT), !is.na(ARRIVAL_DELAY)) |>
```

```

group_by(DESTINATION_AIRPORT) |>
summarise(
  n_aeroporto = n(),
  atrasos = sum(ARRIVAL_DELAY, na.rm = TRUE),
  .groups = "drop") |>
mutate(
  estat_suf = atrasos/n_aeroporto
)
}
flights <- read_csv_chunked("flights.csv.zip",
                           callback = DataFrameCallback$new(funcao),
                           chunk_size = 1e6,
                           col_types = cols_only(
DESTINATION_AIRPORT = col_character(),
ARRIVAL_DELAY = col_double()))

```

2. Selecione a operação apropriada join para incluir, na tabela flights, as colunas CITY e STATE do objeto airports. Para executar esta tarefa:

a. Identifique a coluna que é a chave na tabela flights;

a chave é o destination_airport

b. Identifique a coluna que é a chave na tabela airports;

a chave é o iata_code

c. Quais são os aeroportos que estão listados em flights, mas estão ausentes em airports?

```

faltano <- anti_join(flights, airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE"))
faltano

```

```

# A tibble: 0 x 4
# i 4 variables: DESTINATION_AIRPORT <chr>, n_aeroporto <int>, atrasos <dbl>,
#   estat_suf <dbl>

```

d. Apresente o comando que combine ambas as tabelas, indicando explicitamente as chaves;

e. Armazene a tabela resultante no objeto flights.

```

flights <- right_join(flights, airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE"))
flights

```

```
# A tibble: 1,874 x 8
  DESTINATION_AIRPORT n_aeroporto atrasos estat_suf CITY STATE LATITUDE
  <chr>               <int>      <dbl>      <dbl> <chr>      <chr>      <dbl>
1 ABE                 350      3413        9.75 Allentown PA         40.7
2 ABI                 446      5926       13.3 Abilene TX         32.4
3 ABQ                3241     20448        6.31 Albuquerque NM        35.0
4 ABR                 124      -645       -5.20 Aberdeen SD         45.4
5 ABY                 169      1339        7.92 Albany GA         31.5
6 ACT                 271      4436       16.4 Waco TX         31.6
7 ACV                 294      2748        9.35 Arcata/Eure~ CA        41.0
8 ACY                 701      9069       12.9 Atlantic Ci~ NJ         39.5
9 ADK                  19       -293      -15.4 Adak AK         51.9
10 ADQ                 59        72        1.22 Kodiak AK         57.7
# i 1,864 more rows
# i 1 more variable: LONGITUDE <dbl>
```

3. Quantos aeroportos cada estado possui? Apresente uma tabela ordenada de forma decrescente (no número de aeroportos).

```
n_diacho <- flights |>
  group_by(DESTINATION_AIRPORT) |>
  summarise(
    n_aeroportos = sum(n_aeroporto, na.rm = TRUE)
  ) |>
  arrange(desc(n_aeroportos))

n_diacho
```

```
# A tibble: 322 x 2
  DESTINATION_AIRPORT n_aeroportos
  <chr>               <int>
1 ATL                 343076
2 ORD                 275864
3 DFW                 231764
4 DEN                 193033
5 LAX                 192136
6 SFO                 145409
7 PHX                 145378
8 IAH                 143587
9 LAS                 132124
10 MSP                111146
# i 312 more rows
```

4. Apresente um mapa representando todos os atrasos observados por aeroporto.

a. Carregue o pacote leaflet;

```
library(leaflet)
```

b. Combine os comandos leaflet, addTiles e addMarkers para a criação de um mapa básico;

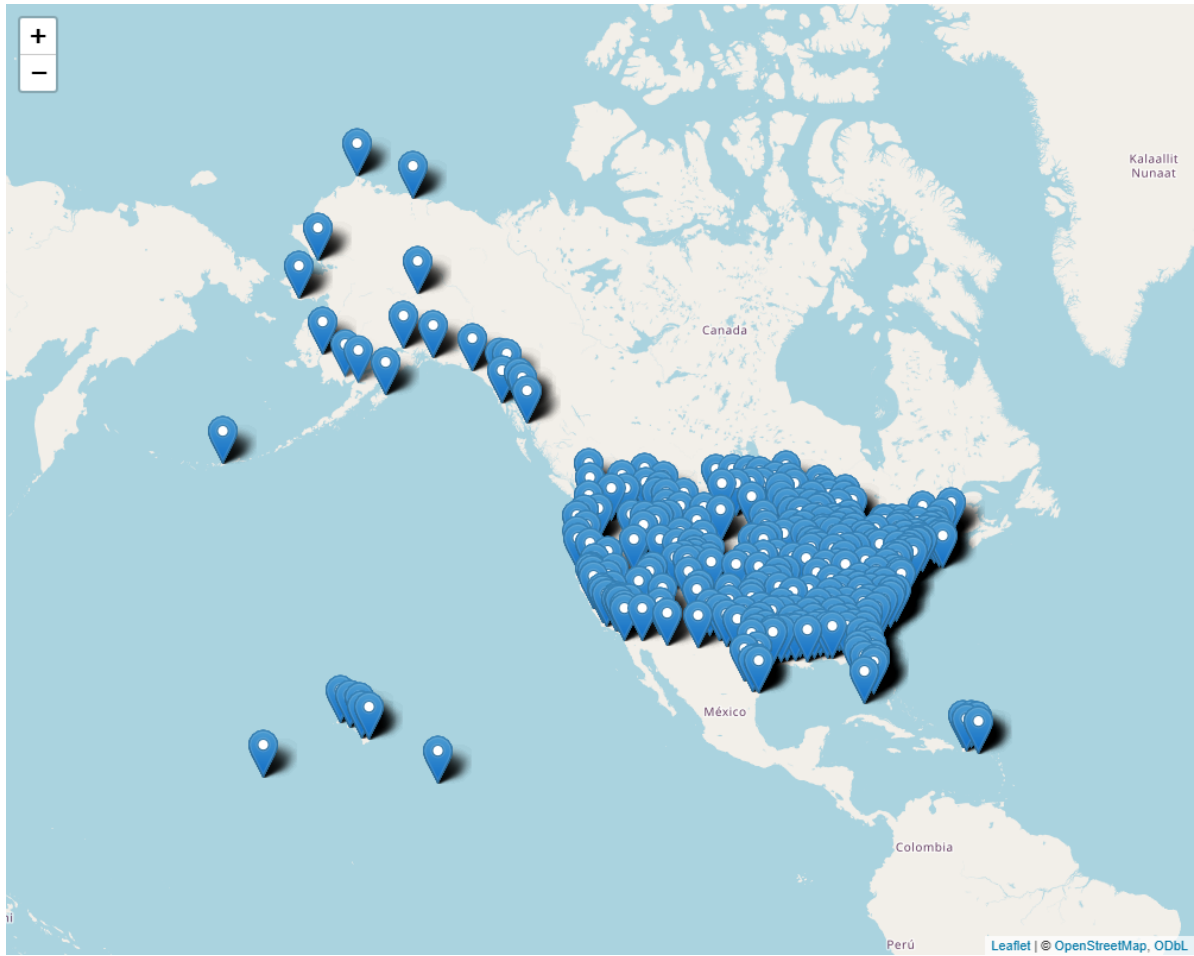
c. Armazene o grafico em b) numa variável chamada this;

```
this <- leaflet(data = flights, width = 1000, height = 800) |>  
  addTiles() |>  
  addMarkers(lng = ~LONGITUDE, lat = ~LATITUDE)
```

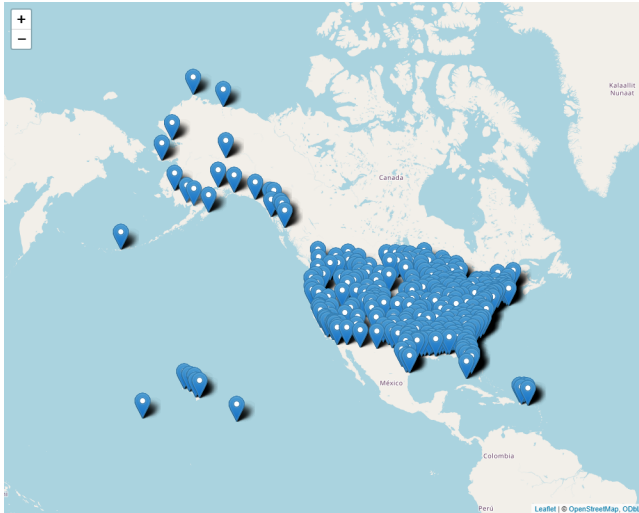
Warning in validateCoords(lng, lat, funcName): Data contains 18 rows with either missing or invalid lat/lon values and will be ignored

```
library(htmlwidgets)  
saveWidget(this, "mapa_temp.html")  
  
library(webshot2)  
webshot("mapa_temp.html", file = "mapa_para_pdf.png",  
        vwidth = 1000, vheight = 800)
```

file:///C:/Users/vitin/OneDrive/Documentos/facul/banco de dados/ME315/mapa_temp.html screens



```
knitr::include_graphics("mapa_para_pdf.png")
```



```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-08-31 21:56:00 -03"
```